

Levantamento: Operação de Cadastro da Diária do Servidor

Não existe um `TramiteController` com método `cadastrar/salvar` isolado. O cadastro da diária do servidor é um fluxo distribuído entre `TramiteController`, `SolicitacaoController` e `ViagemService`, documentado abaixo.

1. Endpoint de entrada

1.1 `TramiteController` (`src/tramite/tramite.controller.ts`)

```
POST /tramite/:id/:nome
@HttpCode(200)
async upsert(
  @Param('id', ParseIntPipe) id: number,
  @Param('nome') nome: string,
  @Body() dto: CreateTramiteDto,
): Promise<UpsertTramiteResponse>
```

- Se `id > 0` → apenas atualiza o trâmite (`tramiteService.update`), não recalcula diárias.
- Se `id <= 0` → **cria** o trâmite (`tramiteService.create(dto, nome)`) e, se `dto.status === 'SOLICITADO'`, dispara o cálculo e persistência das diárias via `viagemService.calcularEPersistirDiarias(created.solicitacao_id)`. É esse disparo que efetivamente gera o cadastro da diária do servidor.

DTO de entrada — `CreateTramiteDto` (`src/tramite/dto/create-tramite.dto.ts`):

```
class CreateTramiteDto {
  cod_lotacao_origem: number;
  lotacao_origem: string;
  cod_lotacao_destino: number;
  lotacao_destino: string;
  status: string; // 'SOLICITADO' | 'RECUSADO' (ver TramiteStatus enum no controller)
  datareg?: Date;
  solicitacao_id: number;
  solicitacao?: solicitacao;
  log_tramite?: log_tramite;
  flag_daof?: string;
}
```

Endpoint dedicado para reprocessar sem criar novo trâmite:

```
POST /tramite/recalcular-diarias/:solicitacaoId (200)
→ viagemService.calcularEPersistirDiarias(solicitacaoId)
```

1.2 `SolicitacaoController` (`src/solicitacao/solicitacao.controller.ts`)

```
POST /solicitacao
create(@Body() createSolicitacaoDto: CreateSolicitacaoDto, @Req() request: Request)
```

Cria apenas o cabeçalho da "solicitação de diária" (registro pai), sem valores calculados. Lê o header `dados_client` (JSON com dados do usuário logado), monta `datareg` (data/hora ajustada por timezone) e `status: 'NAO'`, chama `solicitacaoService.create(solicitacao, usuario)`.

DTO — `CreateSolicitacaoDto` (`src/solicitacao/dto/create-solicitacao.dto.ts`):

```
class CreateSolicitacaoDto {
  datareg: Date;
  justificativa: string;
  status: string;
  cpf_responsavel?: string;
  nome_responsavel?: string;
  cod_lotacao?: number;
  lotacao?: string;
}
```

```

    login?: string;
    protocolo?: string;
    arquivar?: boolean;
  }

```

`SolicitacaoService.create` (`src/solicitacao/solicitacao.service.ts:24`) grava log de auditoria (`logSistemaService.createLog(dto, usuario, Operacao.INSERT)`) e faz `prisma.solicitacao.create(...)`. Não valida cargo/valor — isso ocorre depois, no cálculo da diária.

2. Fluxo completo (ordem de chamadas)

1. Cliente cria a solicitação → `SolicitacaoController.create` → `SolicitacaoService.create` (tabela `solicitacao`).

2. Cliente cadastra evento(s) e participante(s) (módulos `evento`, `participante`, `evento_participantes`) associados à solicitação (pré-requisito, fora deste escopo).

3. Cliente encaminha/tramita a solicitação: `TramiteController.upsert` (POST `/tramite/:id/:nome`, `id=0` para criar) → `TramiteService.create(dto, nome)` (`src/tramite/tramite.service.ts:24`):

- `prisma.tramite.create({ data: dtoSemSolicitacao })` (tabela `tramite`).
 - Se `status === 'SOLICITADO'`: `prisma.solicitacao.update({ where:{id: solicitacao_id}, data:{arquivar:false} })`.
 - Em paralelo (`Promise.all`): `salvarLogTramite` (grava em `log_tramite` via `LogTramiteService`) e `enviarNotificacaoDoStatus` (dispara e-mails via `EmailService`, exceto em `ENV=DEV`).
1. Se `deveCalcular` (status `SOLICITADO`): `ViagemService.calcularEPersistirDiarias(solicitacaoId)` (`src/viagem/viagem.service.ts:165`) — **núcleo do cadastro da diária**:
2. `calculaDiasParaDiaria(solicitacaoId)` (linha 466): busca `prisma.evento.findMany` filtrando `solicitacao_id`, incluindo `evento_participantes` → `participante` e `viagem_participantes` → `viagem`; agrupa eventos por participante (chave = CPF), ordena por data, funde eventos contíguos ($\text{gap} \leq 1$ dia) em "grupos de viagem" e soma `totalDias` por participante usando `Util.totalDeDias`. Retorna `ParticipanteTotalDias[]` com `{participante, totalDias, evento, viagem, eventoParticipanteId}`.
3. Para cada participante elegível (`Promise.allSettled`):
- Se não existe viagem associada, cria uma mínima via `ensureViagemMinima` (`prisma.viagem.create + prisma.viagem_participantes.create`), copiando `pais_id`, `data_ida/volta`, `exterior`, `local_exterior`, `cidade_destino_id` do evento.
 - Chama `calculaDiaria(viagemId, participanteId, eventoId, totalDias, solicitacaoId)` (linha 97) — calcula e grava o valor da diária:
 - Valida `idViagem` (`BadRequestException('Viagem inválida para cálculo de diária')`).
 - Busca `SolicitacaoService.getEventosParticipante(solicitacaoId, participanteId)` para obter `valor_diaria` e lista de eventos do participante.
 - Se não há eventos → `BadRequestException('Nenhum evento encontrado para o participante')`.
 - Se `exterior === 'SIM'` → delega para `calculaInternacional`.
 - Caso nacional: valida `valorDiaria` (`BadRequestException('Valor de diária não encontrado para o cargo do participante')` se nulo/ ≤ 0) e persiste via `ValorViagemService.upsertDiaria` com `{viagem_id, tipo:'DIARIA', destino:'NACIONAL', valor_individual, participante_id}` (tabela `valor_viagem`).
1. Agrega resultado em `ResultadoCalculoDiariasDto { total, elegiveis, calculou, falhas[] }` (`falhas` lista `{participanteId, nome, motivo}` para erros individuais).
2. `TramiteController.upsert` devolve `{ success, calculou, total, elegiveis, falhas }`.

Cálculo por tipo de destino

- **Estadual** (viagem dentro do Amapá, sem passagem aérea):
`CargoDiariaService.findDiariasPorCargo(cargo) + CalculoEstadual.servidores(...)`
(`src/calculo_diarias/estadual.ts`). Viagem para municípios fora de Macapá/Santana/Mazagão paga diária cheia + meia diária extra; viagem "superior a 6 horas" paga meia diária adicional; viagem com pernoite paga diária cheia; valores diferem se `viagem.servidor_acompanhando === 'SIM'`.
- **Nacional** (com passagem aérea, não exterior): `CalculoNacional.servidores(...)`
(`src/calculo_diarias/externo.ts`) usando UF do aeroporto de destino.
- **Internacional** (`evento.exterior === 'SIM'`): `CalculoInternacional.servidores(...)`
(`src/calculo_diarias/internacional.ts`); exige `calculo.valor_diarias` cadastrado para o cargo (`BadRequestException('Valores de diária não cadastrados para o cargo "X"')`); converte USD→BRL via `ValorDiariasService.obterCotacaoDolarVigente()` + `converterUsdParaBrl`; grava diária em USD convertida (`salvaDiariaInternacional`) e, se `tem_passagem === 'SIM'`, também grava a "diária inteira" nacional equivalente (`salvaDiariaInteira`).
- **Macapá**: caso especial de viagem sem passagem dentro de Macapá/AP (retorna 0/null, não gera diária extra).
- Cargo do servidor obtido via `consultaCargo(participanteId)` (linha 573): busca `EventoParticipanteService.findOneParticipante`; se não encontrado → `BadRequestException('Participante X não encontrado')`; se `efetivo === 'SERVIDORES EFETIVOS'` e houver `funcao`, usa a função como cargo; se cargo vazio → `BadRequestException('Participante X sem cargo definido')`.

3. Regras de negócio e validações

- Diária só é calculada/persistida quando o trâmite é criado com `status === 'SOLICITADO'`.
- Participante precisa ter cargo definido (via `funcao` se servidor efetivo), senão `BadRequestException`.
- Cálculo de dias agrupa eventos contíguos ($\text{gap} \leq 1$ dia) por participante, somando dias de viagem consecutivos como um único período de diária.
- Sem participantes elegíveis → retorna `{total:0, elegiveis:0, calculou:false, falhas:[]}` sem lançar erro (apenas warning).
- Falhas de cálculo por participante não interrompem o lote inteiro (`Promise.allSettled`); cada falha é registrada em `falhas[]` com motivo.
- Valor da diária depende do tipo de destino (estadual/nacional/internacional), existência de passagem aérea (`evento.tem_passagem`), pernoite (`viagem.viagem_pernoite`), acompanhamento de servidor (`viagem.servidor_acompanhando`) e cotação do dólar vigente (viagens internacionais).
- `ValorViagemService.upsertDiaria` evita duplicidade: procura `valor_viagem` existente por `viagem_id + tipo='DIARIA' [+ destino] [+ participante_id]`; se existir, atualiza; senão cria.
- Ao criar trâmite com status SOLICITADO, a solicitação é marcada `arquivar:false`.

4. Entidades/tabelas envolvidas (Prisma)

- `solicitacao` — cabeçalho da solicitação de diária.
- `tramite` — histórico de tramitação/aprovação.
- `log_tramite` — log de cada movimentação de trâmite.
- `evento` — evento/viagem-motivo associado à solicitação.

- `evento_participantes` — vínculo participante×evento (contém `cargo`, `funcao`, `efetivo`).
- `participante` — dados do servidor/participante.
- `viagem` — registro de viagem (datas, país, cidade destino, flags `exterior`, `viagem_pernoite`, `viagem_superior`, `servidor_acompanhando`; `tem_passagem` no evento).
- `viagem_participantes` — vínculo participante×viagem.
- `valor_viagem` — **tabela onde a diária calculada é persistida** (tipo: `'DIARIA'`, destino: `'NACIONAL'|'INTERNACIONAL'`, `valor_individual`, `cotacao_dolar`, `justificativa`, `participante_id`, `viagem_id`).
- `valor_diarias` — tabela de referência de valores de diária por cargo/local.
- Também referenciadas: `aeroporto`, `cidade`, `estado`, `pais`, `banco`, `conta_diaria`, `anexo_evento`, `empenho_daofi`, `correcao_solicitacao`.

5. Tratamento de erros/exceções

- `TramiteController.upsert` e `recalcularDiarias` capturam qualquer erro e relançam como `InternalServerErrorException(error.message)` — ou seja, erros de negócio (`BadRequestException`) lançados dentro do serviço podem ser mascarados como 500 ao propagar por esse controller (ponto de atenção / possível bug de design).
- Dentro de `ViagemService`, erros por participante são capturados individualmente em `calcularEPersistirDiarias` via `Promise.allSettled` e reportados em `falhas[]` — não interrompem o processamento dos demais.
- Mensagens específicas:
 - `'Viagem inválida para cálculo de diária'`
 - `'Nenhum evento encontrado para o participante'`
 - `'Valor de diária não encontrado para o cargo do participante'`
 - `'Não foi possível calcular diária internacional (cargo ou regra indisponível)'`
 - `'Valores de diária não cadastrados para o cargo "X"'`
 - `Participante ${id} não encontrado / Participante ${id} sem cargo definido`
 - `Evento ${id} não encontrado / Viagem ${id} não encontrada`

6. Integrações externas

- **E-mail:** `EmailService` (`src/email`) disparado em `TramiteService.enviarNotificacaoDoStatus` para notificar setores (Presidência, ESCOLA, DAOF etc.) conforme `status/cod_lotacao_destino`, exceto em `ENV=DEV`.
- **HTTP externo:** `TramiteController.pesquisaServidor` (`GET /tramite/consulta/detalhes/servidor/:cpf` → `tramiteService.pesquisaServidorGOVBR`) integra com API do GOV.BR (usa `HttpService` do `@nestjs/axios`, injetado no `TramiteService`).
- **Câmbio:** `ValorDiariasService.obterCotacaoDolarVigente()` fornece cotação do dólar para diárias internacionais.
- **Log de auditoria:** `LogSistemaService.createLog` (módulo `log_sistema`) grava operações de INSERT/UPDATE relevantes (ex.: criação de solicitação).

Arquivos de referência

- `src/tramite/tramite.controller.ts`
- `src/tramite/tramite.service.ts`
- `src/tramite/dto/create-tramite.dto.ts`
- `src/solicitacao/solicitacao.controller.ts`
- `src/solicitacao/solicitacao.service.ts`
- `src/solicitacao/dto/create-solicitacao.dto.ts`
- `src/viagem/viagem.controller.ts`
- `src/viagem/viagem.service.ts`
- `src/viagem/calculo-diarias-servidores.ts`
- `src/calculo_diarias/estadual.ts`
- `src/calculo_diarias/externo.ts`
- `src/calculo_diarias/internacional.ts`
- `src/valor_viagem/valor_viagem.service.ts`
- `src/valor_viagem/dto/create-valor_viagem.dto.ts`